# Django Authentication System — Complete Beginner to Advanced Notes

## 1. Authentication System Overview

Authentication হলো ব্যবহারকারীর পরিচয় যাচাই করা। Authorization হলো ব্যবহারকারী কোন কাজ করতে পারবে তা নির্ধারণ করা।

Django authentication-এর প্রধান অংশ:

- Registration
- Login and Logout
- Session Authentication
- Password Hashing
- Permissions and Groups
- Protected Views
- Email/OTP Verification
- Token Authentication
- JWT Authentication
- Custom User Model
- OTP Expiration and Cleanup

### Authentication Flow

```
Registration
    ↓
Email / OTP Verification
    ↓
Account Activation
    ↓
Login
    ↓
Session or Token Issued
    ↓
Access Protected Resources
    ↓
Logout / Token Revocation
```

---

## 2. Project Setup

```
django-admin startproject myauthproject
cd myauthproject
python manage.py startapp accounts
```

Install Django:

```
pip install django
```

For MySQL:

```
pip install mysqlclient
```

For Django REST Framework and JWT:

```
pip install djangorestframework djangorestframework-simplejwt
```

### `settings.py`

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "accounts",
]

LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "profile"
LOGOUT_REDIRECT_URL = "login"
```

Ensure that Django's default middleware includes `SessionMiddleware` and `AuthenticationMiddleware`, and that the authentication and messages context processors are enabled.

### MySQL Configuration

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.mysql",
        "NAME": "myauthdb",
        "USER": "root",
        "PASSWORD": "",
        "HOST": "127.0.0.1",
```

```
        "PORT": "3306",
    }
}
```

Create the database `myauthdb` before migrating. Use environment variables for database credentials in production.

---

## 3. Built-in Authentication: Registration

Django's built-in `UserCreationForm` validates passwords and saves them using Django's password-hashing system.

**`accounts/views.py`**

```
from django.shortcuts import render, redirect
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.decorators import login_required


def signup_view(request):
    if request.method == "POST":
        form = UserCreationForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("login")
    else:
        form = UserCreationForm()

    return render(
        request,
        "accounts/signup.html",
        {"form": form},
    )


@login_required
def profile_view(request):
    return render(request, "accounts/profile.html")
```

**`accounts/urls.py`**

```
from django.urls import path
from django.contrib.auth import views as auth_views
```

```
from . import views

urlpatterns = [
    path("signup/", views.signup_view, name="signup"),

    path(
        "login/",
        auth_views.LoginView.as_view(
            template_name="accounts/login.html"
        ),
        name="login",
    ),

    path(
        "logout/",
        auth_views.LogoutView.as_view(),
        name="logout",
    ),

    path("profile/", views.profile_view, name="profile"),
]
```

**myauthproject/urls.py**

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("accounts.urls")),
]
```

**templates/accounts/signup.html**

```
<h2>Sign Up</h2>

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Sign Up</button>
</form>

<a href="{% url 'login' %}">
```

```
    Already have an account? Login
</a>
```

### `templates/accounts/login.html`

```
<h2>Login</h2>

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Login</button>
</form>

<a href="{% url 'signup' %}">
    Don't have an account? Sign Up
</a>
```

### `templates/accounts/profile.html`

```
<h2>Welcome, {{ request.user.username }}</h2>

<form method="post" action="{% url 'logout' %}">
    {% csrf_token %}
    <button type="submit">Logout</button>
</form>
```

Django's built-in `LogoutView` uses POST for logout in modern Django versions. A POST form is preferable to a logout link.

### Run Migrations

```
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Visit:

- `/accounts/signup/`
- `/accounts/login/`
- `/accounts/profile/`

---

## 4. Session Authentication

Session authentication is Django's standard approach for traditional server-rendered websites.

### Registration View

```
from django.contrib.auth.models import User
from django.shortcuts import render, redirect


def register(request):
    if request.method == "POST":
        username = request.POST.get("username", "").strip()
        email = request.POST.get("email", "").strip()
        password = request.POST.get("password", "")

        if username and email and password:
            if User.objects.filter(username=username).exists():
                return render(
                    request,
                    "register.html",
                    {"error": "Username already exists"},
                )

            User.objects.create_user(
                username=username,
                email=email,
                password=password,
            )

            return redirect("login")

    return render(request, "register.html")
```

For production, prefer a Django form or `UserCreationForm` instead of handling password validation manually.

### Login View

```
from django.contrib.auth import authenticate, login
from django.shortcuts import render, redirect


def user_login(request):
    if request.method == "POST":
        username = request.POST.get("username", "")
        password = request.POST.get("password", "")
```

```
        user = authenticate(
            request,
            username=username,
            password=password,
        )

        if user is not None:
            login(request, user)
            return redirect("profile")

        return render(
            request,
            "login.html",
            {"error": "Invalid username or password"},
        )

    return render(request, "login.html")
```

### Logout View

```
from django.contrib.auth import logout
from django.shortcuts import redirect
from django.views.decorators.http import require_POST


@require_POST
def user_logout(request):
    logout(request)
    return redirect("login")
```

Use either Django's built-in `LogoutView` or this custom view, not both for the same URL.

### Protect a View

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render


@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

### URL Configuration

```
from django.urls import path
from .views import (
    register,
    user_login,
    user_logout,
    dashboard,
)

urlpatterns = [
    path("register/", register, name="register"),
    path("login/", user_login, name="login"),
    path("logout/", user_logout, name="logout"),
    path("dashboard/", dashboard, name="dashboard"),
]
```

A successful call to `login(request, user)` stores authentication information in the session. `logout(request)` clears the current session's authentication state.

---

## 5. Django Permission System

Authentication answers: **Who are you?**

Permissions answer: **What are you allowed to do?**

Django creates default model permissions after migrations:

- `add_model`
- `change_model`
- `delete_model`
- `view_model`

For example, the permission for adding a product in the `shop` app is `shop.add_product`.

### Required Imports

```
from django.contrib.auth.models import Permission, Group
from django.contrib.contenttypes.models import ContentType
```

### Create a Custom Permission

Suppose the project has an `Article` model in the `blog` app.

Prefer declaring custom permissions inside the model:

```
# blog/models.py

from django.db import models
```

```
class Article(models.Model):
    title = models.CharField(max_length=200)
    body = models.TextField()

    class Meta:
        permissions = [
            ("can_publish", "Can Publish Articles"),
        ]
```

Run:

```
python manage.py makemigrations
python manage.py migrate
```

Django will create the custom permission.

Alternatively, permissions can be created programmatically, but ensure that the permission's `ContentType` matches the correct model.

### Assign Permission to a User

```
from django.contrib.auth.models import Permission
from django.contrib.auth import get_user_model

User = get_user_model()

user = User.objects.get(username="faruk")

permission = Permission.objects.get(
    content_type__app_label="blog",
    codename="can_publish",
)

user.user_permissions.add(permission)
```

### Create a Group and Assign Permission

```
from django.contrib.auth.models import Group, Permission

group, _ = Group.objects.get_or_create(name="Editors")

permission = Permission.objects.get(
    content_type__app_label="blog",
    codename="can_publish",
```

```
)

group.permissions.add(permission)
user.groups.add(group)
```

Groups make it easier to manage permissions for multiple users.

### Check Permissions

```
if user.has_perm("blog.can_publish"):
    print("User can publish")
else:
    print("User cannot publish")
```

Check all permissions:

```
user.get_user_permissions()     # Direct user permissions
user.get_group_permissions()    # Permissions from groups
user.get_all_permissions()      # Combined permissions
user.has_module_perms("blog")   # App-level permission check
```

For a user queryset:

```
User.objects.with_perm("blog.can_publish")
```

`with_perm()` is available on Django's user manager; its behavior can depend on the authentication backend and its options.

### Function-Based View (FBV)

```
from django.contrib.auth.decorators import permission_required
from django.shortcuts import render


@permission_required(
    "blog.can_publish",
    raise_exception=True,
)
def publish_article(request):
    return render(request, "publish.html")
```

### Class-Based View (CBV)

```
from django.contrib.auth.mixins import PermissionRequiredMixin
from django.views.generic import TemplateView
```

```
class PublishView(PermissionRequiredMixin, TemplateView):
    template_name = "publish.html"
    permission_required = "blog.can_publish"
    raise_exception = True
```

### Important Permission Notes

- `is_authenticated` indicates whether the request user is authenticated; it does not check a specific permission.
- `is_staff` controls access to the Django admin site, subject to admin permissions.
- `is_superuser` normally bypasses standard permission checks.
- Custom permissions must be enforced on the server, not merely hidden in templates.
- OTP verification and permission checks are separate concerns. Check both if the feature requires both.

---

## 6. Custom Token Authentication Without DRF

A custom token system can be useful for a small API, but it requires you to implement token expiry, revocation, secure storage and request authentication correctly.
For larger APIs, consider Django REST Framework authentication classes or a well-maintained token solution.

### Token Model

```
# accounts/models.py

import hashlib
import secrets

from django.conf import settings
from django.db import models
from django.utils import timezone


class UserToken(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )
    token_hash = models.CharField(max_length=64, unique=True)
```

```
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Token for {self.user_id}"

    @staticmethod
    def hash_token(raw_token):
        return hashlib.sha256(
            raw_token.encode("utf-8")
        ).hexdigest()

    @classmethod
    def issue(cls, user):
        raw_token = secrets.token_urlsafe(32)
        token_hash = cls.hash_token(raw_token)

        cls.objects.update_or_create(
            user=user,
            defaults={"token_hash": token_hash},
        )

        # Return the raw token only once.
        return raw_token
```

The database stores a hash rather than the raw token. If the database is exposed, this reduces the risk of immediate token reuse.

### Login and Token Generation

```
# accounts/token_views.py

from django.contrib.auth import authenticate
from django.http import JsonResponse
from django.views.decorators.http import require_POST
from django.views.decorators.csrf import csrf_protect

from .models import UserToken


@require_POST
@csrf_protect
def token_login(request):
    username = request.POST.get("username", "")
    password = request.POST.get("password", "")
```

```
    user = authenticate(
        request,
        username=username,
        password=password,
    )

    if user is None:
        return JsonResponse(
            {"error": "Invalid credentials"},
            status=401,
        )

    raw_token = UserToken.issue(user)

    return JsonResponse({"token": raw_token})
```

This form-based example is CSRF-protected. If the login endpoint is intended for a non-browser API client, use an appropriate API authentication and CSRF strategy rather than disabling CSRF indiscriminately.

### Token Authentication Decorator

```
# accounts/decorators.py

from functools import wraps

from django.http import JsonResponse
from django.utils import timezone

from .models import UserToken


def token_required(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        authorization = request.headers.get(
            "Authorization", ""
        )

        scheme, separator, raw_token = authorization.partition(" ")

        if scheme.lower() != "bearer" or not separator or not raw_token:
            return JsonResponse(
```

```
                {"error": "Bearer token required"},
                status=401,
            )

        token_hash = UserToken.hash_token(raw_token)

        try:
            token_obj = UserToken.objects.select_related(
                "user"
            ).get(token_hash=token_hash)
        except UserToken.DoesNotExist:
            return JsonResponse(
                {"error": "Invalid token"},
                status=401,
            )

        if not token_obj.user.is_active:
            return JsonResponse(
                {"error": "Inactive account"},
                status=401,
            )

        request.user = token_obj.user
        request.auth_token_obj = token_obj

        return view_func(request, *args, **kwargs)

    return wrapper
```

This is a minimal example and does not implement expiry yet. Add a token expiry field and reject expired tokens before using it in production.

### Protected API

```
# accounts/token_views.py

from django.http import JsonResponse
from .decorators import token_required


@token_required
def token_profile(request):
    return JsonResponse({
        "username": request.user.username,
```

```
        "email": request.user.email,
    })
```

Send the token using:

```
Authorization: Bearer YOUR_TOKEN
```

### Token Logout / Revocation

```
from django.http import JsonResponse
from django.views.decorators.http import require_POST
from .decorators import token_required


@require_POST
@token_required
def token_logout(request):
    request.auth_token_obj.delete()

    return JsonResponse({
        "message": "Logged out successfully"
    })
```

Because the example uses a one-token-per-user model, issuing a new token replaces the previous token. For multiple devices, use a separate token record for each session/device.

---

## 7. Custom User Model

A custom user model is useful when the project needs additional identity fields or different login behavior.

**Important:** Set `AUTH_USER_MODEL` before the project's first migration. Changing user models after production data exists can require a complex migration.

**`accounts/models.py`**

```
from django.db import models
from django.contrib.auth.models import AbstractUser


class User(AbstractUser):
    email = models.EmailField(unique=True)
    phone = models.CharField(
        max_length=15,
```

```
        unique=True,
        null=True,
        blank=True,
    )
    is_verified = models.BooleanField(default=False)
```

### `settings.py`

```
AUTH_USER_MODEL = "accounts.User"
```

Then run:

```
python manage.py makemigrations accounts
python manage.py migrate
```

Use `get_user_model()` or `settings.AUTH_USER_MODEL` instead of hardcoding Django's default `User` in reusable models and application code.

### Why Prefer `AbstractUser`?

`AbstractUser` preserves Django's standard username, password, staff, active and permission fields, along with its compatible authentication behavior.

If using `AbstractBaseUser` instead, you must also configure a custom user manager, `USERNAME_FIELD`, `REQUIRED_FIELDS`, and the necessary permission/staff fields. That approach is more advanced.

---

## 8. OTP Verification

An OTP (One-Time Password) is a temporary code used to verify a user or authorize a specific action.

A secure OTP implementation should have:

- A short expiration time.
- A maximum number of attempts.
- A hashed code instead of a stored plain-text code.
- Rate limiting for code requests.
- Single-use verification.
- A secure delivery channel.
- Cleanup of expired records.

### OTP Model

```
# accounts/models.py
```

```
from django.conf import settings
from django.db import models
from django.utils import timezone


class OTP(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )
    otp_hash = models.CharField(max_length=64)
    salt = models.CharField(max_length=32)
    attempts = models.PositiveSmallIntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)
    consumed_at = models.DateTimeField(null=True, blank=True)

    def is_expired(self):
        return (
            timezone.now() - self.created_at
        ).total_seconds() > 300
```

## Generate and Hash an OTP

```
# accounts/otp_utils.py

import hashlib
import hmac
import secrets


def generate_otp():
    # Six-digit code, including possible leading zeros.
    return f"{secrets.randbelow(1_000_000):06d}"


def hash_otp(code, salt):
    return hashlib.sha256(
        f"{salt}:{code}".encode("utf-8")
    ).hexdigest()


def verify_otp_hash(code, salt, stored_hash):
    calculated = hash_otp(code, salt)
```

```
    return hmac.compare_digest(
        calculated,
        stored_hash,
    )
```

Because a six-digit OTP has low entropy, hashing alone is not enough to protect against a database attacker trying every possible code. Short expiry, strict attempt limits and rate limiting are essential. For higher assurance, use a keyed hash with a secret held outside the database.

### Create an OTP Record

```
import secrets

from .models import OTP
from .otp_utils import generate_otp, hash_otp


def create_otp(user):
    code = generate_otp()
    salt = secrets.token_hex(16)

    OTP.objects.filter(
        user=user,
        consumed_at__isnull=True,
    ).delete()

    OTP.objects.create(
        user=user,
        otp_hash=hash_otp(code, salt),
        salt=salt,
    )

    # Deliver `code` through email/SMS in a real application.
    # Never return the OTP to a public API response.
    return code
```

Call this function from a rate-limited endpoint and send the returned code through a trusted email/SMS provider. Do not log real OTPs in production.

### Verify an OTP

```python
from django.db import transaction
from django.utils import timezone

from .models import OTP
from .otp_utils import verify_otp_hash


@transaction.atomic
def verify_user_otp(user, code):
    otp = (
        OTP.objects
        .select_for_update()
        .filter(
            user=user,
            consumed_at__isnull=True,
        )
        .order_by("-created_at")
        .first()
    )

    if otp is None or otp.is_expired():
        return False

    if otp.attempts >= 5:
        return False

    otp.attempts += 1
    otp.save(update_fields=["attempts"])

    if not verify_otp_hash(
        code,
        otp.salt,
        otp.otp_hash,
    ):
        return False

    otp.consumed_at = timezone.now()
    otp.save(update_fields=["consumed_at"])

    user.is_verified = True
    user.is_active = True
    user.save(update_fields=["is_verified", "is_active"])
```

```
    return True
```

The example activates the account after successful verification. Adapt the logic if OTP is for password reset, login, or a sensitive transaction instead.
For reliable concurrent attempt limiting, ensure the OTP record is locked in a transaction and consider database-level throttling. Apply rate limits to both sending and verification endpoints.

---

## 9. JWT Authentication with Django REST Framework

JWT (JSON Web Token) is commonly used for APIs. It is different from Django's session authentication and from the custom token example above.

### Install Packages

```
pip install djangorestframework djangorestframework-simplejwt
```

**`settings.py`**

```
INSTALLED_APPS = [
    # Django apps...
    "rest_framework",
    "accounts",
]

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ],
}
```

### JWT Login View

```
# accounts/api_views.py

from django.contrib.auth import authenticate
from rest_framework.decorators import (
    api_view,
    permission_classes,
)
from rest_framework.permissions import AllowAny
from rest_framework.response import Response
from rest_framework import status
```

```
from rest_framework_simplejwt.tokens import RefreshToken


@api_view(["POST"])
@permission_classes([AllowAny])
def jwt_login(request):
    username = request.data.get("username", "")
    password = request.data.get("password", "")

    user = authenticate(
        request,
        username=username,
        password=password,
    )

    if user is None or not user.is_active:
        return Response(
            {"error": "Invalid credentials"},
            status=status.HTTP_401_UNAUTHORIZED,
        )

    refresh = RefreshToken.for_user(user)

    return Response({
        "refresh": str(refresh),
        "access": str(refresh.access_token),
    })
```

`RefreshToken.for_user()` does not automatically reject inactive users, so the view explicitly checks `user.is_active`.

### JWT URLs

```
# accounts/urls.py

from django.urls import path
from .api_views import jwt_login

urlpatterns = [
    path("api/token/", jwt_login, name="jwt_login"),
]
```

For a standard Simple JWT setup, you can also use its built-in views for obtaining, refreshing and verifying tokens:

```
from django.urls import path
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
    TokenVerifyView,
)

urlpatterns += [
    path("api/token/", TokenObtainPairView.as_view(),
name="token_obtain_pair"),
    path("api/token/refresh/", TokenRefreshView.as_view(),
name="token_refresh"),
    path("api/token/verify/", TokenVerifyView.as_view(),
name="token_verify"),
]
```

Use either your custom login endpoint or the built-in obtain-pair endpoint at a given URL, not both.

### Protect an API

```
from rest_framework.decorators import (
    api_view,
    permission_classes,
)
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response


@api_view(["GET"])
@permission_classes([IsAuthenticated])
def api_profile(request):
    return Response({
        "username": request.user.username,
        "email": request.user.email,
    })
```

Client request:

```
GET /accounts/api/profile/
Authorization: Bearer YOUR_ACCESS_TOKEN
```

### JWT Security Notes

- Access tokens should have a short lifetime.
- Refresh tokens should be protected and rotated/revoked according to the application's needs.
- A JWT is not encrypted by default; do not put secrets in its payload.
- A stateless access token generally remains valid until expiry unless the API adds a revocation strategy.
- Do not mix session, custom token and JWT authentication without a clear design.

---

## 10. Celery: Automatically Delete Expired OTPs

Celery can run scheduled background tasks to clean up expired OTP records.

Install the packages:

```
pip install celery redis
```

This example assumes the project package is named `myauthproject` and Redis is available.

**`myauthproject/celery.py`**

```
import os

from celery import Celery

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "myauthproject.settings",
)

app = Celery("myauthproject")
app.config_from_object(
    "django.conf:settings",
    namespace="CELERY",
)
app.autodiscover_tasks()
```

**`myauthproject/__init__.py`**

```
from .celery import app as celery_app

__all__ = ("celery_app",)
```

### OTP Cleanup Method

Add the following method to the `OTP` model:

```
from datetime import timedelta
from django.utils import timezone


@classmethod
def cleanup_expired(cls):
    cutoff = timezone.now() - timedelta(minutes=5)

    deleted, _ = cls.objects.filter(
        created_at__lt=cutoff,
    ).delete()

    return deleted
```

Ensure this method is indented inside the `OTP` model class. If you use an expiry field or lockout timestamp, include those conditions in the cleanup query as appropriate.

### `accounts/tasks.py`

```
from celery import shared_task
from .models import OTP


@shared_task
def delete_expired_otps():
    return OTP.cleanup_expired()
```

### `settings.py`

```
CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"

CELERY_BEAT_SCHEDULE = {
    "delete-expired-otps-every-minute": {
        "task": "accounts.tasks.delete_expired_otps",
        "schedule": 60.0,
    },
}
```

### Run Celery

In separate terminals:

```
celery -A myauthproject worker --loglevel=info
```

```
celery -A myauthproject beat --loglevel=info
```

Make sure Redis is running. Celery Beat schedules the task; the Celery worker executes it. For a small application, Django management commands or database retention jobs may be sufficient.

---

## 11. Django Admin Integration

For a custom user model based on `AbstractUser`, you can extend the standard admin configuration.

```
# accounts/admin.py

from django.contrib import admin
from django.contrib.auth.admin import UserAdmin
from .models import User


@admin.register(User)
class CustomUserAdmin(UserAdmin):
    list_display = (
        "username",
        "email",
        "phone",
        "is_verified",
        "is_staff",
        "is_active",
    )
    search_fields = ("username", "email", "phone")
```

If the project uses Django's default user model, do not register a second user model unnecessarily. Register the model actually configured by `AUTH_USER_MODEL`.

---

## 12. Important Security Best Practices

1. Use Django's password hashers and built-in authentication forms.
2. Keep `SECRET_KEY`, database passwords and API secrets outside source control.
3. Require HTTPS in production.
4. Use CSRF protection for browser-based session authentication.

5. Keep authentication and authorization checks on the server.
6. Add expiry, attempt limits and rate limiting to OTP verification.
7. Do not store plain-text OTPs or raw long-lived tokens in the database.
8. Use `secrets`, not `random`, for security-sensitive token generation.
9. Avoid `@csrf_exempt` unless the API's authentication and CSRF threat model justifies it.
10. Never trust a user ID, permission name or account status submitted by the client.
11. Use `request.user.is_authenticated` for login checks and `has_perm()` for permission checks.
12. Use generic login errors and rate limiting to reduce account enumeration and brute-force risk.
13. Keep Django, DRF, Simple JWT and other dependencies updated.
14. Test login, logout, expired tokens, invalid OTPs, inactive users and unauthorized access.
15. Use audit logging for important security events without logging passwords, tokens or OTP codes.

---

## 13. Authentication Methods Compared

| Feature | Session | Custom Token | JWT |
|---|---|---|---|
| Common use | Django websites | Custom APIs | REST APIs |
| Server-side state | Yes | Usually yes | Access token can be stateless |
| Browser cookies | Common | Optional | Optional |

| | | | |
|---|---|---|---|
| Revocation | Clear session | Delete token record | Requires expiry/blacklist strategy |
| Complexity | Low | Medium to high | Medium |
| Recommended starting point | Standard Django site | Only for a specific need | DRF API requirements |

---

## 14. Interview Questions and Answers

**Q1. What is authentication?**

Authentication verifies the identity of a user.

**Q2. What is authorization?**

Authorization determines what an authenticated user can access or perform.

**Q3. How does Django store passwords?**

Django stores password hashes rather than raw passwords and verifies submitted passwords with its configured password hashers.

**Q4. What is the difference between `authenticate()` and `login()`?**

`authenticate()` checks credentials and returns a user if valid. `login()` establishes the authenticated session.

**Q5. What does `login_required` do?**

It redirects unauthenticated users to the configured login URL by default.

**Q6. What is a Django permission?**

A permission controls whether a user may perform a particular action, such as `blog.can_publish`.

**Q7. What is the difference between groups and permissions?**

Permissions represent allowed actions. Groups bundle permissions and assign them to multiple users.

**Q8. What is a session?**

A session stores server-side user state and is typically referenced by a session cookie in the browser.

**Q9. What is the difference between a token and a JWT?**

A custom token is an application-defined credential. A JWT is a structured signed token containing claims. Both require secure handling and expiry policies.

**Q10. Why hash OTPs?**

Hashing helps avoid storing the raw code in the database, although a short numeric OTP still needs strict expiry and attempt limits.

**Q11. Why use `AUTH_USER_MODEL`?**

It allows the project to specify its user model and is particularly important when custom identity fields are required.

**Q12. What is Celery Beat?**

Celery Beat schedules recurring tasks, while a Celery worker executes them.

---

## 15. Quick Revision

```
Registration:
    UserCreationForm / create_user()

Password verification:
    authenticate()

Session login:
    login(request, user)

Session logout:
    logout(request)

Protect website view:
    @login_required

Check permission:
    user.has_perm("blog.can_publish")

Protect CBV:
    PermissionRequiredMixin

Custom user:
    AUTH_USER_MODEL = "accounts.User"
```

```
Custom token:
    secrets.token_urlsafe() + expiry + revocation

OTP:
    secure generation + hash + expiry + attempt limits

JWT:
    Simple JWT + short-lived access token + protected API

Scheduled cleanup:
    Celery Beat + Celery worker
```

**Final recommendation:** For a normal Django website, start with Django's built-in authentication and sessions. Add groups and permissions when needed. For APIs, use Django REST Framework with an appropriate authentication strategy. Add OTP verification only when the application needs email/phone verification or another one-time verification flow.